

W5D1 - bash basics

La parte principale dell'esercizio di questa sezione è stato svolto nello script presente in questa cartella, `bash_basics.sh`.

Questo pdf contiene alcuni esercizi facoltativi.

W, WHO e WHOAMI

Il comando `w` riporta chi è loggato nel sistema e quello che sta facendo

Il comando `who` riporta semplicemente chi è loggato nel sistema

Il comando `whoami` riporta il nome effettivo dell'utente che lo lancia.

```
danix @ FlyingDutchman { dom lug 27 } [ ~ ]
1 $> whatis w
w (1) - Show who is logged on and what they are doing.
danix @ FlyingDutchman { dom lug 27 } [ ~ ]
2 $> whatis who
who (1) - show who is logged on
danix @ FlyingDutchman { dom lug 27 } [ ~ ]
3 $> whatis whoami
whoami (1) - print effective user name
danix @ FlyingDutchman { dom lug 27 } [ ~ ]
4 $> w
 16:29:01 up  5:53,  1 user,  load average: 0,76, 0,66, 0,65
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
danix     tty7     :0               10:35    5:53m 13:32   1.68s i3
danix @ FlyingDutchman { dom lug 27 } [ ~ ]
5 $> who
danix     tty7           2025-07-27 10:35 (:0)
danix @ FlyingDutchman { dom lug 27 } [ ~ ]
6 $> whoami
danix
danix @ FlyingDutchman { dom lug 27 } [ ~ ]
7 $> █
```

MAN

Il comando `man` consente di leggere il manuale per la maggior parte dei comandi presenti su linux. Quando chiediamo il manuale per un comando, `man` cerca quel comando in diverse sezioni, così definite:

Sezione	Contenuto
1	Programmi eseguibili e comandi della shell
2	Chiamate di sistema (funzioni fornite dal kernel)
3	Chiamate di libreria (funzioni fornite dalle librerie di programma)

Sezione	Contenuto
4	Files speciali (di solito quelli che troviamo in <code>/dev</code>)
5	Formati di files e convenzioni (ad esempio <code>/etc/passwd</code>)
6	Giochi
7	Miscellanea (macro pacchetti e convenzioni, ad esempio <code>man(7)</code> o <code>man-pages(7)</code>)
8	Comandi per l'amministrazione di sistema (di solito disponibili solo per root)
9	Routines del kernel (non standard)

Se invece si specifica la sezione, il comando `man` andrà a cercare direttamente nella sezione specificata.

Ad esempio il semplice comando `man man` ci riporterà la pagina di manuale per il comando `man`, mentre `man 7 man` ci riporterà la pagina di manuale per `groff man`, un sistema di formattazione di pagine che permette di creare le pagine di man che vediamo ogni giorno.

1 | `man job`

```
danix @ FlyingDutchman { dom lug 27 } [ ~ ]
7 $> man job
Non c'è il manuale per job
danix @ FlyingDutchman { dom lug 27 } [ ~ ]
8 $> 
```

Quando non esiste una pagina man per un comando vediamo un messaggio di errore nella shell.

1 | `man ps`

```

PS(1)                                     General Commands Manual                                     PS(1)

NAME
  ps - report a snapshot of the current processes.

SYNOPSIS
  ps [option ...]

DESCRIPTION
  ps displays information about a selection of the active processes.  If you want a repetitive update of the selection and the displayed information, use top instead.

  This version of ps accepts several kinds of options.

  • Unix options, which may be grouped and must be preceded by a dash.

  • BSD options, which may be grouped and must not be used with a dash.

  • GNU long options, which are preceded by two dashes.

  Options of different types may be freely mixed, but conflicts can appear.  There are some synonymous options, which are functionally identical, due to the many standards and ps implementations that this ps is compatible with.

  By default, ps selects all processes with the same effective user ID (euid=EUID) as the current user and associated with the same terminal as the invoker.  It displays the process ID (pid=PID), the terminal associated with the process (tname=TTY), the cumulated CPU time in [DD-]hh:mm:ss format (time=TIME), and the executable name (ucmd=CMD).  Output is unsorted by default.

  The use of BSD-style options will add process state (stat=STAT) to the default display and show the command args (args=COMMAND) instead of the executable name.  You can override this with the PS_FORMAT environment variable.  The use of BSD-style options will also change the process selection to include processes on other terminals (TTYs) that are owned by you; alternately, this may be described as setting the selection to be the set of all processes filtered to exclude processes owned by other users or not on a terminal.  These effects are not considered when options are described as being "identical" below, so -H will be considered identical to Z and so on.

  Except as described below, process selection options are additive.  The default selection is discarded, and then the selected processes are added to the set of processes to be displayed.  A process will thus be shown if it meets any of the given selection criteria.

EXAMPLES

```

1 | `man kill`

```

KILL(1)                                     User Commands                                     KILL(1)

NAME
  kill - terminate a process

SYNOPSIS
  kill [-signal|-s signal|-p] [-q value] [-a] [--timeout milliseconds signal] [--] pid|name...

  kill -l [number|0xsigmask] | -L

  kill -d pid

DESCRIPTION
  The command kill sends the specified signal to the specified processes or process groups.

  If no signal is specified, the TERM signal is sent.  The default action for this signal is to terminate the process.  This signal should be used in preference to the KILL signal (number 9), since a process may install a handler for the TERM signal in order to perform clean-up steps before terminating in an orderly fashion.  If a process does not terminate after a TERM signal has been sent, then the KILL signal may be used; be aware that the latter signal cannot be caught, and so does not give the target process the opportunity to perform any clean-up before terminating.

  Most modern shells have a builtin kill command, with a usage rather similar to that of the command described here.  The --all, --pid, and --queue options, and the possibility to specify processes by command name, are local extensions.

  If signal is 0, then no actual signal is sent, but error checking is still performed.

ARGUMENTS
  The list of processes to be signaled can be a mixture of names and PIDs.

  pid
    Each pid can be expressed in one of the following ways:

    n
      where n is larger than 0.  The process with PID n is signaled.

    0
      All processes in the current process group are signaled.

    -1

```

VI(M)

`vi` è l'editor di testo per eccellenza su GNU/Linux, è presente dal 1976 su UNIX e lo troviamo ancora oggi più o meno su tutte le distribuzioni (a volte con nomi diversi come vim, elvis, vile), infatti se siamo in difficoltà su un sistema sconosciuto, è probabile che possiamo leggere un file con vi.

Nell'esempio seguente si vedrà l'interfaccia di [neovim](#), un clone moderno di vim (vi improved) che è a sua volta un clone di vi con più funzionalità.

```
1 | vim pippo
```

```
1 |
```



ps e kill di un processo

Quando un processo diventa irresponsivo, si blocca e non ci permette di utilizzarlo, o semplicemente occupa risorse inutilmente, sarà necessario "killarlo", ovvero rimuoverlo forzatamente usando il comando `kill` o una delle sue varianti.

Per utilizzare il comando `kill`, dobbiamo conoscere il `PID` del programma che vogliamo chiudere, quindi utilizzeremo il comando `ps` per visualizzare i processi attivi, e da lì, trovato il PID del comando che ci interessa potremo eliminarlo

```
1 | ps axu
```

```
danix 16791 0.1 0.3 579312 84796 ? S1 12:46 0:22 /usr/lib64/libexec/kactivitymanagend
root 18900 0.0 0.0 0 0 ? I 16:27 0:00 [kworker/u16:0-events_unbound]
root 19726 0.0 0.0 0 0 ? I 16:27 0:00 [kworker/2:0-ata_sff]
danix 20788 0.0 0.1 63228 30840 ? S 16:28 0:00 urxvt
danix 20789 0.0 0.0 13292 9004 pts/6 Ss 16:28 0:00 bash
danix 20787 0.0 0.0 7088 2140 ? Ss 16:28 0:00 /usr/bin/dbus-daemon --syslog --fork --print-pid 5 --print-address 7 --session
danix 21296 0.0 0.0 165664 6912 ? S1 12:19 0:00 /usr/libexec/gvfsd-metadata
danix 21641 0.0 0.0 20264 11840 ? S 10:51 0:00 /usr/libexec/speech-dispatcher-modules/sd_espeak-ng /etc/speech-dispatcher/modules/espeak-ng.conf
danix 21654 0.0 0.0 358644 6812 ? S1 10:51 0:01 /usr/libexec/speech-dispatcher-modules/sd_dumwg /etc/speech-dispatcher/modules/dumwg.conf
danix 21658 0.0 0.0 20332 11872 ? S 10:51 0:00 /usr/libexec/speech-dispatcher-modules/sd_espeak-ng /etc/speech-dispatcher/modules/
danix 21680 0.0 0.0 111964 7432 ? Ssl 10:51 0:00 /usr/bin/speech-dispatcher --spawn --communication-method unix_socket --socket-path /run/user/1000/speech-dispatcher/speechd.sock
danix 22371 3.9 1.9 2365316 314432 ? Ssl 16:29 0:14 gimp
danix 22449 0.0 0.1 224332 31728 ? S1 16:29 0:00 /usr/lib64/gimp/3.0/plugins/script-fu/script-fu -gimp 277 15 14 -run 0
danix 23734 0.0 0.0 4224 2576 ? S 16:30 0:00 clipnotify
root 23750 0.0 0.0 0 0 ? I 16:30 0:00 [kworker/0:0-ata_sff]
root 24225 0.0 0.0 0 0 ? I< 16:31 0:00 [kworker/u17:2]
root 24445 0.0 0.0 0 0 ? I 16:31 0:00 [kworker/5:0-ata_sff]
root 25081 0.0 0.0 0 0 ? I 16:32 0:00 [kworker/1:0-ata_sff]
danix 25749 0.0 0.0 8976 6772 pts/1 S+ 16:32 0:00 man scrot
danix 25787 0.0 0.0 3688 2740 pts/1 S+ 16:32 0:00 less
root 27093 0.0 0.0 0 0 ? I 14:52 0:01 [kworker/2:1-i915-events_unordered]
root 27353 0.0 0.0 0 0 ? I 16:33 0:00 [kworker/u16:3-events_unbound]
danix 28242 0.0 0.0 7088 2056 ? Ss 14:53 0:00 /usr/bin/dbus-daemon --syslog --fork --print-pid 5 --print-address 7 --session
danix 28584 0.0 0.0 13604 9764 pts/6 S+ 16:34 0:00 vim pippo
danix 28585 0.3 0.1 13480 16640 ? Ss 16:34 0:00 vim --embed pippo
danix 28900 0.0 0.0 7088 2048 ? Ss 16:35 0:00 /usr/bin/dbus-daemon --syslog --fork --print-pid 5 --print-address 7 --session
danix 28932 0.0 0.0 3368 2272 ? S 16:35 0:00 sleep 50
danix 29201 0.0 0.0 20020 14240 ? S 16:35 0:00 xterm -class WXTerm -u8
danix 29206 0.1 0.0 13224 8980 pts/7 Ss 16:35 0:00 bash
danix 29214 0.0 0.0 7088 2176 ? Ss 16:35 0:00 /usr/bin/dbus-daemon --syslog --fork --print-pid 5 --print-address 7 --session
danix 29537 0.0 0.0 3368 2280 pts/5 S+ 16:35 0:00 sleep 5
danix 29606 0.0 0.0 3368 2296 ? S 16:35 0:00 sleep 1
danix 29622 0.0 0.0 7904 5188 pts/7 R+ 16:35 0:00 ps axu
root 29867 0.0 0.0 0 0 ? D< 14:27 0:01 [kworker/u17:1-i915_flip]
root 30703 0.0 0.0 0 0 ? I 16:09 0:00 [kworker/u16:2-events_unbound]
danix @ FlyingButchman { dom lug 27 } █
2 => kill -9 28564
danix @ FlyingButchman { dom lug 27 } █
3 =>
```

```
1 | kill -9 28564
```

come vediamo nello screenshot qui sopra, il comando `ps axu` ci ha restituito una serie di processi in esecuzione, tra cui il nostro `vim pippo` con PID 28564. Col comando successivo abbiamo detto a kill di inviare il segnale 9 a vim, segnale che indica al programma di terminare immediatamente il processo.

I segnali più comuni sono elencati in questa tabella:

Segnale	Numero	Descrizione
SIGHUP	1	Indica che il terminale è stato chiuso.
SIGINT	2	Interruzione del processo (solitamente Ctrl+C).
SIGQUIT	3	Interruzione del processo e generazione di un core dump.
SIGILL	4	Istruzione non valida.
SIGABRT	6	Abort del processo (chiamata a abort()).
SIGFPE	8	Errore aritmetico (es. divisione per zero).
SIGKILL	9	Termina il processo immediatamente (non può essere catturato).
SIGTERM	15	Richiesta di terminazione del processo (catturabile).
SIGSTOP	19	Ferma il processo (non può essere catturato).

Sono presenti molti altri segnali, e sono tutti documentati nel man leggibile con `man 7 signal`.

BACKGROUND e FOREGROUND

Quando avviamo un programma, questo normalmente tende ad occupare la nostra shell, non permettendoci quindi di fare nient'altro fino a che non lo avremo chiuso, ma ci sono programmi che non hanno bisogno della shell per funzionare e per interagire con l'utente, come i programmi che hanno un'interfaccia grafica.

In questi casi, possiamo lanciare il comando in background, quindi l'esecuzione verrà sganciata dalla shell in cui è stato lanciato il programma e questo continuerà ad operare senza che noi lo vediamo. La sintassi per lanciare un programma in background richiede di mettere un **&** alla fine del comando in questo modo:

```
1 | /usr/bin/firefox &
```

In questo modo il programma verrà lanciato e noi avremo la shell libera per lanciare un altro programma o fare qualsiasi altra cosa dobbiamo fare.

Se per qualche motivo avessimo bisogno di recuperare il programma lanciato in background, potremmo utilizzare il comando `jobs` per vedere quale programma intendiamo recuperare, e il comando `fg %n` riprendere l'output del programma desiderato (n in questo caso è un numero >1).

Ad esempio, abbiamo già firefox in background, dal comando precedente. Supponiamo adesso di lanciare dolphin (un file browser) sempre in background e poi ci ricordiamo che ci serve leggere l'output di firefox.

Questo è quello che faremmo nella shell:

```
1 | /usr/bin/dolphin &
2 | jobs
3 | [1]-  In esecuzione          firefox &
4 | [2]+  In esecuzione          dolphin &
```

A questo punto per riprendere l'output di firefox lanceremo il comando:

```
1 | fg %1
```

e firefox tornerà in foreground. A questo punto potremo terminarlo con un semplice `Ctrl+c`.

Spazio su disco

dalla shell è possibile visualizzare una miriade di informazioni sul sistema su cui stiamo operando.

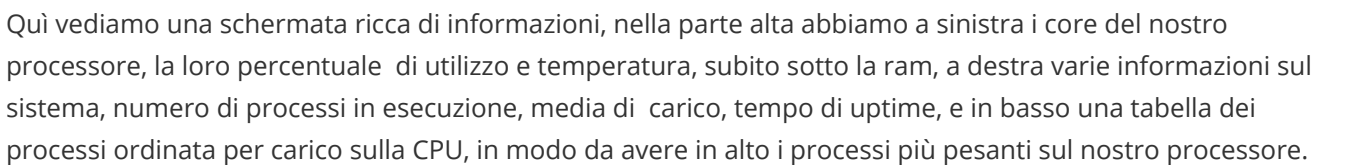
Ad esempio per visualizzare lo spazio occupato su disco (o dischi) possiamo utilizzare il comando `df` (disk free). Questo comando riporta lo spazio disponibile dell'unità disco comunicata come argomento, se non è presente un argomento, df visualizzerà lo spazio disponibile di tutti i dischi presenti sulla macchina. Ecco un esempio di output:

```
1 | /usr/bin/df -h
2 | File system          Dim. Usati Dispon. Uso% Montato su
3 | tmpfs                 3,9G  1,7M    3,9G   1% /run
4 | devtmpfs              8,0M    0    8,0M   0% /dev
5 | /dev/sda2             219G   45G   164G  22% /
6 | none                  320K   26K   290K   8% /sys/firmware/efi/efivars
7 | tmpfs                 7,8G  2,2M    7,8G   1% /dev/shm
8 | cgroup_root           8,0M    0    8,0M   0% /sys/fs/cgroup
9 | /dev/sdb1             246G   96G   137G  42% /home
10 | /dev/sdb2             670G  398G   239G  63% /data
11 | /dev/sda1             100M   24M    76M  25% /boot/efi
12 | 172.16.34.11:/Volume1/Slackware 3,6T  1,9T   1,6T  54% /mnt/slackware
13 | 172.16.34.4:/mnt/HD/HD_a2/share 915G  220G   696G  25% /mnt/shared
14 | 172.16.34.4:/mnt/HD/HD_a2/danix 915G  220G   696G  25% /mnt/backup_danix
15 | 172.16.34.11:/Volume1/Library 3,6T  1,9T   1,6T  54% /mnt/Media
16 | tmpfs                 7,8G    0    7,8G   0% /var/lib/pkgtools/setup/tmp
17 | tmpfs                 1,6G   11M    1,6G   1% /run/user/1000
18 |
```

l'opzione `-h` passata a `df` imposta la visualizzazione delle unità di misura su potenze di 1024 invece del default (numero di blocchi) che risulterebbe poco leggibile.

htop

Un modo più "grafico" per avere molte info sul nostro sistema da riga di comando consiste nell'utilizzare un'applicazione come `top` o similari. Nello screenshot seguente vedremo [htop](#).



Alla prossima. 😊